

Point distribution (48 points total):

- **Exercise 1 — Core Game (14 pts):** A playable JavaFX game with a solid OOP class hierarchy.
- **Exercise 2 — State Machine NPCs (12 pts):** NPC behaviour implemented as finite state machines.
- **Exercise 3 — Database Integration (12 pts):** Game data loaded from a database via DAO.
- **Reflection (10 pts):** A short written reflection on your experience using AI for this project.
- **Bonus — Multiplayer / Networking (+4 pts):** Multiplayer or a REST-based online leaderboard.

Extra homework drop: Students who submit a **genuinely playable game** covering **Exercises 1, 2, and 3 as well as the Reflection** may additionally have their single lowest regular assignment (A01–A07) excluded from grade calculation — on top of the standard Streichübung. Contact your instructor after submission to request this.

This assignment combines a **large programming project** with a short **reflection** on using AI coding tools. You are required to use an AI assistant (GitHub Copilot is free for students via GitHub Education). Claude, Cursor, Windsurf and all others are also valid. Your `AGENTS.md` and at least one `FEATURE.md` must be included in your submission ZIP. They are evidence of your context engineering process.

Requirements

- You **must** use an AI assistant to help build the project.
 - It **must** be a Java 25+ Maven project that compiles and runs.
 - **Any game is valid** — build whatever you want, as long as it covers the four required features (OOP, JavaFX, FSM NPCs, DAO + Database). The dungeon crawler below is a concrete starting point for those without an idea.
 - `AGENTS.md` and at least one `FEATURE.md` must be in the ZIP root.
-

Game Description (for those without an idea)

Build a **top-down dungeon crawler / roguelike**. The player explores a tile-based dungeon, fights enemies, trades with a merchant, and tries to escape. The game ties together every major topic from this semester.

Core features:

- **2D tile grid dungeon:** rooms and corridors on a grid (static or procedurally generated). Tile types: Floor, Wall, Door.
- **Player character:** moves with WASD / arrow keys; has health points and a simple inventory (weapon, potion, key).

- **JavaFX rendering:** tile grid on a Canvas; HUD showing HP and score; coloured rectangles or tile sprites.
- **Enemy NPCs with FSMs:**
 - *Goblin*: PATROL → CHASE (player nearby) → ATTACK (adjacent) → FLEE (low HP)
 - *Boss*: IDLE → PHASE_1 ($\leq 50\%$ HP) → PHASE_2 ($\leq 25\%$ HP)
- **Merchant NPC:** IDLE → TALKING (player interacts) → TRADING; dialogue lines loaded from the database.
- **Database:** items (name, description, effect, price), NPC dialogue lines, and a high-score table; all loaded via DAO at runtime.

Nice-to-have (optional):

- Procedurally generated maps or multiple dungeon levels
- Fog of war / line-of-sight
- Save / load game state
- Additional NPC types (archer, mage, trap)
- Basic multiplayer via network or a REST-based online leaderboard (counts for the bonus)

The project is intentionally large. Spend as long as you have fun on it, starting from Session 8 with the basics and adding FSM and DAO support after Sessions 9 and 10.

Exercise 1 (14 points) Exercise 1 — Core Game: OOP + JavaFX

Build a playable JavaFX game with a well-structured OOP class hierarchy. This exercise covers the foundations that the rest of the project builds on.

Part A — Playable game (4 pts): The game must run and be interactable. The player can act according to your game rules, there is a win or lose condition, and the game reaches it without crashing. (It does not have to be feature-complete — a working core loop with a clear end state is sufficient.)

Part B — OOP architecture (5 pts): Your code must demonstrate the OOP skills from Sessions 1–4:

- An entity class hierarchy (e.g. abstract Entity → Player, NPC, Item) with meaningful inheritance and @Override
- At least one interface used across multiple classes (e.g. Interactable, Renderable)
- All fields private; encapsulation enforced
- Packages used to organise the code (3 layer architecture at minimum)

Part C — JavaFX rendering (5 pts):

- Tile map rendered on a Canvas using GraphicsContext
- At least one HUD element (e.g. HP bar, score display)
- Keyboard input handled via setOnKeyPressed / setOnKeyReleased
- The game is visually coherent and the player can see the game world

Submission: Include in your PDF a screenshot of the running game and a brief architecture note (3–5 sentences describing your class hierarchy and package structure). Your AGENTS.md and at least one FEATURE.md must be in the ZIP root.

Exercise 2 (12 points) Exercise 2 — State Machine NPCs

Implement at least two NPC types whose behaviour is driven by **finite state machines (FSMs)** as covered in the State Machines exercise (yes you can re-use the code from the exercise).

Part A — FSM implementation (6 pts):

- At least 2 NPC types with distinct FSMs (e.g. enemy + merchant, or two enemy types with different states)
- States defined as an enum; transitions triggered by game events (player distance, HP threshold, interaction)
- FSM `update()` method called once per game tick; the NPC's behaviour changes visibly when the state changes

Part B — Demonstration (3 pts): Show the FSM in action. Include in your PDF either a sequence of screenshots (at least 3, one per state) or a short console transcript showing state transitions (e.g. “Goblin: PATROL → CHASE (player at distance 3)”).

Part C — Written description (3 pts): For each NPC type, include a **state table** or state diagram in your PDF:

State	Behaviour	Transition trigger
PATROL	Moves along a fixed path	Player enters detection range
CHASE	Moves toward player	Player adjacent / low HP
...	...	...

Exercise 3 (12 points) Exercise 3 — Database Integration (DAO Pattern)

Add database persistence using the DAO pattern and 3-layer architecture.

Part A — DAO layer (5 pts):

- Define at least 2 DAO interfaces (e.g. `ItemDao`, `DialogueDao`, `ScoreDao`)
- Provide a JDBC implementation for each interface
- Use prepared statements throughout; no SQL string concatenation
- Implement at least `findAll()` and one other CRUD operation (e.g. `save()` for scores)

Part B — Runtime data loading (4 pts): At least one entity type must be loaded from the database at game startup — not hardcoded in Java:

- Items with their stats (name, description, effect, price), or
- NPC dialogue lines, or
- anything else that makes sense for your game

Include a **database schema** (CREATE TABLE statements or an ERD) in your PDF.

Part C — 3-layer architecture (3 pts): Your code must maintain a clean separation of layers:

- No SQL in `model/` or `controller/` or `view/` classes
- No game logic in DAO implementations
- A brief architecture note in your PDF (3–5 sentences) explaining how the layers interact and which classes belong to each layer

SQLite is an accepted alternative to MariaDB. If you use SQLite, include the `.db` file in the ZIP or provide a setup script that creates and seeds the database.

Exercise 4 (+4 Bonus points) Bonus — Multiplayer / Networking

Add network capability (again the below are just examples, you can implement anything that makes sense for your game):

- **LAN multiplayer:** Two players connect on the same network. Minimum: each player sees the other's position on the map.
- **REST leaderboard:** Use Java `HttpClient` to post scores to and retrieve them from a simple REST API (your own server or a public test API). Display the top-5 scores in the game UI.

Implementing either option earns the full 4 bonus points. Include a brief description in your PDF of how the networking is implemented and how to test it.

Reflection (10 points)

This is THE ONLY PART of this exercise that you need to do manually. Everything else, including the documentation of your game can be done with AI.

Write a short reflection covering your experience using AI tools on this project. Aim for **1–2 pages** (roughly 400–700 words). Quality and honesty matter more than length — a few generic sentences earn fewer points than a shorter but concrete account.

Cover the following in any order or format (paragraphs or bullet points):

1) Your project

What did you build? What features are implemented, and what is missing or unfinished — and why?

2) How you used AI

How did you work with AI tools? What did you ask the AI to do, what did you do yourself, and which modes (Ask / Edit / Agent / Plan, or equivalents) were most useful for which kinds of tasks?

3) Prompting and context

Give one example of a request that worked well and one that did not. Explain what you changed to improve it. Did adding context (e.g. AGENTS.md, naming classes, adding constraints) help?

4) Verification and testing

How did you check AI-generated code? What bugs did you catch, and how? Did you use git to save working states before accepting large changes?

5) Problems and limits

What was your biggest problem? Give one concrete example where AI helped you and one where it did not help (or suggested wrong solutions), and describe what you did.

6) Code quality and trust

How well do you understand the code you submitted? Was it easy to read and change, or did it become hard to maintain? Did your trust in AI-generated code change during the project?

7) Time and counterfactual

How many hours did you spend? Could you have built something similar without AI, and how long would it have taken?

8) Next time

Would you use AI again for a larger project? What would you do differently — in prompting, verification, or project planning?

Submit your reflection as part of your PDF, together with the game screenshots and architecture notes.

Submission Requirements

- Read the *Organisational Guidelines* on Moodle and follow them.
- **PDF:** ALD_A08_LASTNAME.pdf — contains: reflection, architecture note, class hierarchy overview, state diagrams/tables (Ex 2), database schema (Ex 3), and screenshots or transcripts.
- **ZIP:** ALD_A08_LASTNAME.zip — contains: the complete Maven project (all .java files, pom.xml), AGENTS.md, at least one FEATURE.md. If using SQLite, include the .db file or a setup script.
- Include a brief “how to run” note in the PDF if the project requires special setup (e.g. starting a local MariaDB instance, running a seed script).
- Naming convention: 1 point deduction for incorrect file names.